

Taylor expansion for Imperative Programs

Guy McCusker (University of Bath)

joint work with
Pierre Clairambault and Lionel Vaux-Auclair
(Aix-Marseille Université)

Taylor Expansion of Lambda Terms

Ehrhard and Regnier (2003) introduced the *Differential Lambda Calculus* and with it the *Taylor expansion of lambda-terms*.

$$Px = \sum_{n=0}^{\infty} \frac{1}{n!} (D^n P \cdot x^n)0$$

$D^n P \cdot x^n$ represents a version of P *linearly applied* to n occurrences of its argument.

The *fully developed* Taylor expansion — in which subterms are recursively expanded — can be seen as an infinite sum of *linear* terms drawn from a *resource calculus*.

Differential lambda-calculus

This has led to a great deal of further developments, including:

- abstract categorical theory of *differential* and *tangent* categories e.g. (Blute, Cockett and Seely)
- comparisons with Gödel's *Dialectica* interpretation (Kerjean and Pédrot)
- redevelopment of the classical theory of lambda-calculus using Taylor expansion (Barbarossa and Manzonetto)
- the 2024 Alonzo Church Award.

In this talk:

- Some curiosity-driven exploration of resource calculus and Taylor expansion
- A new view on imperative programs

Resource Calculus

The resource calculus replaces regular application of lambda-terms MN with a resource-sensitive application:

$$m, n ::= x \mid \lambda x. m \mid m\bar{n}$$

$$\bar{n} ::= [n_1, \dots, n_k] \quad \text{a finite multiset of terms}$$

Intuition: execution of MN is like execution of $M[N, \dots, N]$ for *some* number of copies of N .

Resource calculus reduction

$$(\lambda x. m)\bar{n} \rightarrow_{\beta} m[\bar{n}/x]$$

Linear substitution generates a finite *sum* of terms:

$$x[\bar{n}/x] = \begin{cases} n & \text{if } \bar{n} = [n] \\ \mathbf{0} & \text{otherwise} \end{cases}$$

$$y[\bar{n}/x] = \begin{cases} y & \text{if } \bar{n} = [] \\ \mathbf{0} & \text{otherwise} \end{cases}$$

$$(\lambda y. m)[\bar{n}/x] = \lambda y. m[\bar{n}/x]$$

$$(m\bar{p})[\bar{n}/x] = \sum_{\bar{n}=\bar{n}_1 \uplus \bar{n}_2} m[\bar{n}_1/x] \bar{p}[\bar{n}_2/x]$$

$$([p_1, \dots, p_k])[\bar{n}/x] = \sum_{\bar{n}=\bar{n}_1 \uplus \dots \uplus \bar{n}_k} [p_1[\bar{n}_1/x], \dots, p_k[\bar{n}_k/x]]$$

All syntactic operations can be extended to sums via *linearity*. This includes preservation of $\mathbf{0}$, so any zero that appears destroys the whole term.

(Technically we're working in the free module over the terms, with coefficients drawn from a suitable semiring.)

Resource calculus behaviour

Reduction is *strongly normalizing* and (locally) *confluent*.

Terms can experience *starvation* e.g.

$$(\lambda x. x[x])([\lambda x. x[x], \lambda x. x[x]]) \rightarrow (\lambda x. x[x])[\lambda x. x[x]] \rightarrow \mathbf{0}$$

Surfeit also destroys a term:

$$(\lambda x. x[x])([\lambda x. x[x], \lambda x. x[x], \lambda x. x[x]]) \rightarrow \mathbf{0}$$

Taylor Expansion

The Taylor expansion of a lambda-term is now readily defined:

$$\mathbf{T}(x) = x$$

$$\mathbf{T}(\lambda x. M) = \lambda x. \mathbf{T}(M)$$

$$\mathbf{T}(MN) = \sum_k \frac{1}{k!} \mathbf{T}(M) [\mathbf{T}(N)^k]$$

(If using an idempotent semiring — which suffices for our purposes — the rational coefficient can be ignored.)

Key result: Böhm commutation

Ehrhard and Regnier (2008) demonstrated that normalised Taylor expansions coincide with Taylor-expanded Böhm trees:

$$NF(\mathbf{T}(M)) = \mathbf{T}(\mathbf{BT}(M)).$$

So equivalence of Taylor expansions is the same as equivalence of Böhm trees.

Barbarossa and Manzonetto (2019) used this to redevelop several classical lambda-calculus results using Taylor expansions.

Taylor expansion provides a powerful new approximation theory for the lambda-calculus.

A semantic view: Relational Semantics

These ideas grew from *quantitative semantics* of lambda-calculus and linear logic (Girard, 1988).

The simplest variant of quantitative semantics is the *relational model*:

- types = sets
- $\llbracket A \rightarrow B \rrbracket = \mathbf{M}_f \llbracket A \rrbracket \times \llbracket B \rrbracket$
- terms $x : A \vdash M : B$ = relations between $\mathbf{M}_f A$ and B

(Reddy, 1996) described a relational model of *imperative* programs based on lists rather than multisets.

From multisets to lists

A (natural?) question: can we adapt the resource calculus and associated theory to a *noncommutative* setting, replacing *multisets* with *lists*?

(We remark that (Tsukada, Asada and Ong, 2017) introduced a “rigid” resource calculus, using lists, but subject to permutations; we abandon the permutations and embrace the fixed ordering.)

First attempt

The syntax looks the same as before, thanks to the magic of overloading.

$$m, n ::= x \mid \lambda x. m \mid m \vec{n}$$

$$\vec{n} ::= [n_1, \dots, n_k] \quad \text{a finite list of terms}$$

$$(\lambda x. m) \vec{n} \rightarrow_{\beta} m[\vec{n}/x]$$

What is the substitution?

Noncommutative substitution

$$x[\vec{n}/x] = \begin{cases} n & \text{if } \vec{n} = [n] \\ \mathbf{0} & \text{otherwise} \end{cases}$$

$$y[\vec{n}/x] = \begin{cases} y & \text{if } \vec{n} = [] \\ \mathbf{0} & \text{otherwise} \end{cases}$$

$$(\lambda y. m)[\vec{n}/x] = \lambda y. m[\vec{n}/x]$$

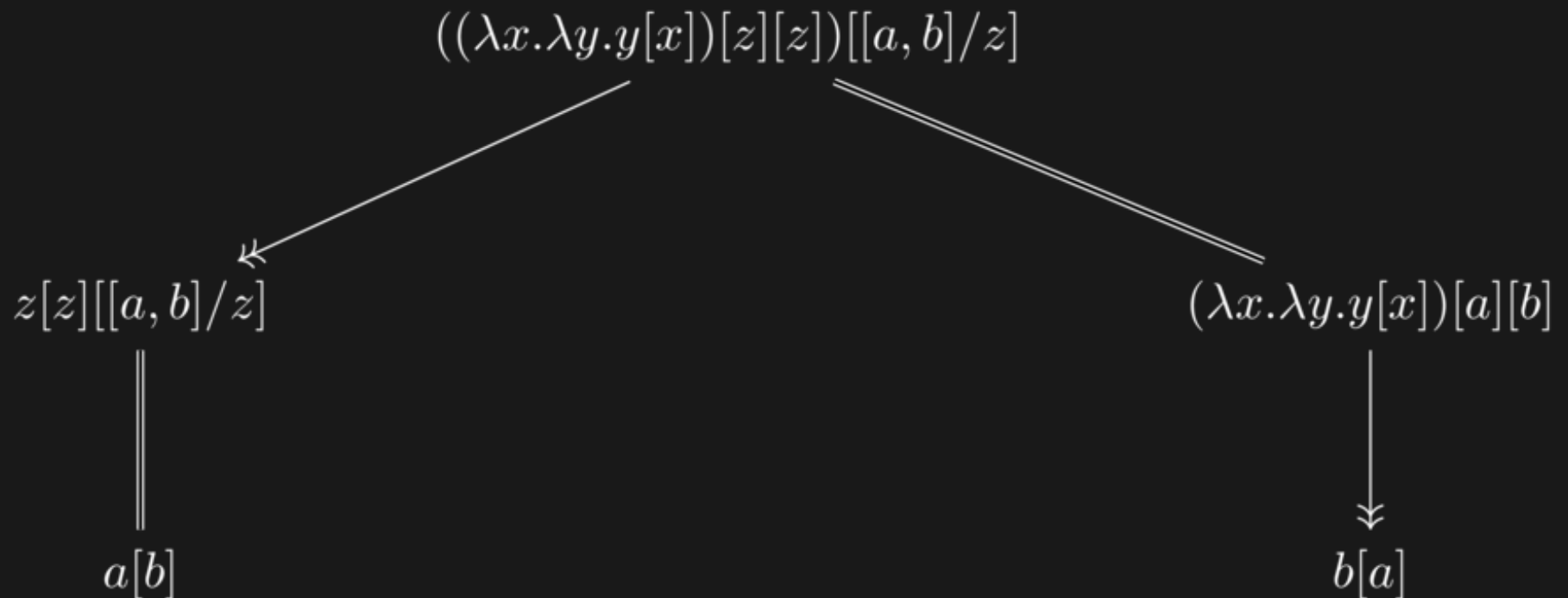
$$(m\vec{p})[\vec{n}/x] = \sum_{\vec{n} = \vec{n}_1 \cdot \vec{n}_2} m[\vec{n}_1/x] \vec{p}[\vec{n}_2/x]$$

$$([p_1, \dots, p_k])[\vec{n}/x] = \sum_{\vec{n} = \vec{n}_1 \cdot \dots \cdot \vec{n}_k} [p_1[\vec{n}_1/x], \dots, p_k[\vec{n}_k/x]]$$

In contrast to the original resource calculus, the “sums” here are either $\mathbf{0}$ or singletons.

Problem: confluence fails

Confluence is immediately problematic:



A heavy-handed solution

We can avoid this situation by restricting our syntax.

A *linear* application rule

$$\frac{\Gamma \vdash m \quad \Delta \vdash n_i \quad (i = 1, \dots, k)}{\Gamma, \Delta \vdash m[n_1, \dots, n_k]}$$

prevents variables from occurring in both function and argument.

The resulting calculus is very boring: just linear lambda-calculus.
But it is confluent!

Solution (more interesting)

Perhaps we can restore some interesting computations to the calculus by adding a *sequencing* construct

$$m ; n$$

with an *additive* style rule

$$\frac{\Gamma \vdash m \quad \Gamma \vdash n}{\Gamma \vdash m ; n}$$

A noncommutative resource calculus

We can add *constants* and *assertions* to obtain a noncommutative resource calculus.

$$m, n ::= x \mid c \mid \lambda x. m \mid m \vec{n} \mid m ; n \mid \text{assert}_c m$$

Assertions will allow us to encode conditionals using formal sums:

$$\text{assert}_{\text{tt}} B ; M + \text{assert}_{\text{ff}} B ; N$$

behaves like if-then-else.

Linear substitution

Linear substitution now works as expected.

Multiple occurrences of a variable are always separated by ; or occur in separate parts of a list: there is a well-defined order.

Linear substitution respects this order.

Resource calculus reductions

$$(\lambda x. m)\vec{n} \rightarrow_{\beta} m[\vec{n}/x] \qquad \text{skip} ; m \rightarrow m$$

$$\text{assert}_c c \rightarrow \text{skip} \qquad \text{assert}_c d \rightarrow \mathbf{0}$$

$$\text{assert}_c m ; n \rightarrow m ; \text{assert}_c n$$

$$\frac{m \rightarrow m'}{m ; n \rightarrow m' ; n} \qquad \frac{n \rightarrow n'}{m ; n \rightarrow m ; n'}$$

A clue: relational semantics of imperative programs
(Reddy, 1996) pioneered a relational semantics of *imperative* programs.

His model replaces the multisets of the familiar relational model of linear logic with lists: a non-commutative relational semantics.

It was later shown [M 2002] that this model is *fully abstract* for a programming language called *Basic SCI*.

Reddy's work was an important precursor to the game semantics of local variables [Abramsky & M, 1996].

SCI: Syntactic Control of Interference

Reynolds (1978) introduced *syntactic control of interference* (SCI) as an approach to control of *aliasing* in imperative programming.

Key idea: functions and their arguments should not share identifiers.

Reynolds used this property to ensure that distinct identifiers never *interfere*; this enables cleaner reasoning about programs.

Nowadays we see this as a *linear (or affine) type system*.

SCI is a restricted *Idealized Algol*. It has an important place in the development of programming language theory.

- O'Hearn's early work on SCI led him to notice the phenomenon *doubly-closed categories*
- ... which led to Bunched Implications (O'Hearn and Pym, 1999)
- ... and hence to Separation Logic (Reynolds, 2002)

Basic SCI

Basic SCI is a higher-order imperative programming language with local variables and an affine type system.

Types:

$\mathbf{X} ::= \mathbf{U} \mid \mathbf{B} \mid \mathbf{N}$ unit, Booleans, natural numbers

$A ::= \mathbf{X} \mid A \rightarrow B$

Constants:

$$k ::= \text{skip} \mid \text{tt} \mid \text{ff} \mid i$$

Syntax:

$$\begin{aligned} M, N, P ::= & x \mid \lambda x^A. M \mid MN \\ & \mid l := M \mid !l \mid M ; N \mid \text{while } M \text{ do } N \mid \text{new } l \text{ in } M \\ & \mid k \mid \text{isZero } M \mid \text{pred } M \mid \text{succ } M \mid \text{if } M \text{ then } N \text{ else } P \end{aligned}$$

l ranges over an infinite set of *locations*.

Note that there is no recursion: a function cannot call itself because

$$f(f(\dots))$$

violates the linearity constraint.

Type judgements $\Gamma \vdash^L M : A$:

$$\frac{\Gamma \vdash^L M : A \rightarrow B \quad \Delta \vdash^{L'} N : A}{\Gamma, \Delta \vdash^{L, L'} MN : B}$$

$$\frac{\Gamma \vdash^L M : \mathbf{U} \quad \Gamma \vdash^L N : \mathbf{X}}{\Gamma \vdash^L M ; N : \mathbf{X}}$$

$$\frac{\Gamma \vdash^{L, l} M : \mathbf{X}}{\Gamma \vdash^L \text{new } l \text{ in } M : \mathbf{X}}$$

Operational semantics

This language can be given a small-step operational semantics:

$$M, \sigma \rightarrow M', \sigma'$$

where σ, σ' are *states*: functions from locations to integers.

Key reductions:

$$(\lambda x. M)N, \sigma \rightarrow M[N/x], \sigma$$

$$l := i, \sigma \rightarrow \text{skip}, \sigma[l \mapsto i]$$

$$!l, \sigma \rightarrow \sigma(i), \sigma$$

$$\text{new } l \text{ in } M, \sigma \rightarrow M, \sigma[l \mapsto 0]$$

In the last case the set of available locations grows to include l .

From SCI to resource calculus

We'll now see how to extend our noncommutative resource calculus to incorporate the ideas from Reddy's model.

The corresponding Taylor expansion will map SCI terms to sums of resource terms, following the ideas of Reddy's interpretation.

Imperative resource terms

Resource calculus (reminder):

$$m, n ::= x \mid c \mid \lambda x. m \mid m \vec{n} \mid m ; n \mid \text{assert}_c m$$

To this we add assignment and lookup operations:

$$m, n ::= \dots \mid !l \mid l := i$$

Note that the more general $l := m$ can be encoded via sums and asserts as

$$\sum_i \text{assert}_i m ; l := i$$

Imperative resource reductions

We add no more reductions to the calculus!

The calculus is *strongly normalizing* and (locally) confluent.

So how does it model state? Via the Taylor expansion of local variables.

Imperative Taylor Expansion

We can begin to give the Taylor expansion of SCI terms.

A term M will be taken to a formal sum $\mathbf{T}(M)$ of resource terms.

For the lambda-calculus part, the definition appears identical to Ehrhard and Regnier's:

$$\begin{aligned}\mathbf{T}(x) &= x \\ \mathbf{T}(\lambda x. M) &= \lambda x. \mathbf{T}(M) \\ \mathbf{T}(MN) &= \sum_k \mathbf{T}(M)[\mathbf{T}(N)^k]\end{aligned}$$

It is *not* the same — it is much dumber!

- we work with lists rather than multisets
- in the application MN there are no identifiers in common between M and N
- the deterministic nature of our substitution means that no rational coefficients are required, even if we use a non-idempotent semiring

Arithmetic and conditionals are handled using sequencing, assert, and sums:

$$\begin{aligned} \mathbf{T}(\text{pred } M) &= \text{assert}_0 \mathbf{T}(M) ; 0 \\ &\quad + \sum_{i \in \mathbb{N}} \text{assert}_{i+1} \mathbf{T}(M) ; i \\ \mathbf{T}(\text{succ } M) &= \sum_{i \in \mathbb{N}} \text{assert}_i \mathbf{T}(M) ; i + 1 \end{aligned}$$

$$\begin{aligned} \mathbf{T}(\text{if } M \text{ then } N \text{ else } P) &= \text{assert}_{\text{tt}} \mathbf{T}(M) ; \mathbf{T}(N) \\ &\quad + \text{assert}_{\text{ff}} \mathbf{T}(M) ; \mathbf{T}(P) \end{aligned}$$

Basic imperative constructs:

$$\mathbf{T}(M ; N) = \mathbf{T}(M) ; \mathbf{T}(N)$$

$$\mathbf{T}(! l) = ! l$$

$$\mathbf{T}(l := M) = \sum_{i \in \mathbb{N}} \text{assert}_i \mathbf{T}(M) ; l := i$$

We just need to figure out variable allocation...

Motivating example

Consider the term

$$f : \mathbf{U} \rightarrow \mathbf{U} \vdash^l f(l := !l + 1) ; !l : \mathbf{N}$$

The first terms of its Taylor expansion are

$$\begin{aligned} & f([]) ; !l \\ + & \sum_i f[\text{assert}_i !l ; l := i + 1] ; !l \\ + & \sum_{i,j} f[\text{assert}_i !l ; l := i + 1, \text{assert}_j !l ; l := j + 1] ; !l \\ + & \dots \end{aligned}$$

(after simplifying some assertions.)

Each term contains a single trace of reads and writes to location l .

The behaviour of such a term after l is allocated to a cell will be given by:

- replacing each $! l$ with an appropriate constant, reflecting the last value written
- replacing each $l := i$ with skip

We can perform this replacement using a variant of linear substitution.

Semantics of local variables

We define an operation of linear substitution of *state-traces* into resource terms.

State events are sequences of operations:

$$op ::= w(i) \mid r(i)$$

Given a *trace* t — a sequence of these operations — and a location l , we define the substitution

$$m\langle t/l \rangle$$

on resource terms.

This substitution represents the result of term m , executed in an environment where location l insists on performing the trace t .
(Compare this to a linear application to a fixed list of inputs.)

The key cases:

$$(! l') \langle t/l \rangle := \begin{cases} i & \text{if } t = r(i) \text{ and } l = l' \\ ! l' & \text{if } t = \varepsilon \text{ and } l \neq l' \\ \mathbf{0} & \text{otherwise} \end{cases}$$

$$(l' := i) \langle t/l \rangle := \begin{cases} \text{skip} & \text{if } t = w(i) \text{ and } l = l' \\ l' := i & \text{if } t = \varepsilon \text{ and } l \neq l' \\ \mathbf{0} & \text{otherwise} \end{cases}$$

$$c\langle t/l \rangle := \begin{cases} c & \text{if } t = \varepsilon \\ \mathbf{0} & \text{otherwise} \end{cases}$$

$$x\langle t/l \rangle := \begin{cases} x & \text{if } t = \varepsilon \\ \mathbf{0} & \text{otherwise} \end{cases}$$

$$(\lambda x. m)\langle t/l \rangle := \lambda x. m\langle t/l \rangle$$

$$(m\vec{n})\langle t/l \rangle := \begin{cases} m\langle t/l \rangle\vec{n} & \text{if } l \text{ is a location of } m \\ m\vec{n}\langle t/l \rangle & \text{if } l \text{ is a location of } n \end{cases}$$

$$[m_1, \dots, m_k]\langle t/l \rangle := \sum_{t=t_1 \cdots t_k} [m_1\langle t_1/l \rangle, \dots, m_k\langle t_k/l \rangle]$$

$$(m ; n)\langle t/l \rangle := \sum_{t=t_1 \cdot t_2} m\langle t_1/l \rangle ; n\langle t_2/l \rangle$$

$$(\text{assert}_c m)\langle t/l \rangle := \text{assert}_c m\langle t/l \rangle$$

Taylor expansion of variable allocation

Variable allocation is expanded via substitution of all possible *valid traces*: those where the reads and writes match up as you would expect.

$$\mathbf{T}(\text{new } l \text{ in } M) = \sum_{t \in \text{Val}} \mathbf{T}(M)\langle t/l \rangle.$$

Back to our example

Recall the term

$$f : \mathbf{U} \rightarrow \mathbf{U} \vdash^l f(l := !l + 1) ; !l : \mathbf{N}$$

with its Taylor expansion

$$\begin{aligned} & f([]) ; !l \\ + & \sum_i f[\text{assert}_i !l ; l := i + 1] ; !l \\ + & \sum_{i,j} f[\text{assert}_i !l ; l := i + 1, \text{assert}_j !l ; l := j + 1] ; !l \\ + & \dots \end{aligned}$$

To compute the expansion of

$$f : \mathbf{U} \rightarrow \mathbf{U} \vdash^{\emptyset} \text{new } l \text{ in } f(l := !l + 1) ; !l : \mathbf{N}$$

we perform all possible substitutions with valid state-traces, and discard any $\mathbf{0}$ s that arise.

When f does not invoke its argument, the location is left unchanged:

$$\sum_{t \in \text{Val}} (f(\square) ; !l) \langle t/l \rangle = \sum_{t = r(0)} (f(\square) ; !l) \langle t/l \rangle = f(\square) ; 0$$

When f invokes its argument once, the location is incremented once:

$$\sum_{t \in \text{Val}} \left(\sum_i f[\text{assert}_i ! l ; l := i + 1 ; ! l] \langle t/l \rangle \right) = \sum_i f[\text{assert}_i 0 ; \text{skip}] ; i + 1$$

because the only traces yielding a non-zero substitution are of the form

$$r(0)w(i + 1)r(i + 1).$$

Discarding failed asserts, this is

$$f[\text{assert}_0 0 ; \text{skip}] ; 1$$

which reduces to

$$f[\text{skip}] ; 1$$

When f invokes its argument twice, the location is incremented twice:

$$\begin{aligned} & \sum_{t \in \text{Val}} \left(\sum_{i,j} f[\text{assert}_i ! l ; l := i + 1, \text{assert}_j ! l ; l := j + 1] ; ! l \right) \langle t/l \rangle \\ &= \sum_{t \in \text{Val}} \sum_{i,j} f[\text{assert}_i 0 ; \text{skip}, \text{assert}_j i + 1 ; \text{skip}] ; j + 1 \end{aligned}$$

This reduces to

$$f[\text{assert}_0 0 ; \text{skip}, \text{assert}_1 1 ; \text{skip}] ; 2$$

and then to

$$f[\text{skip}, \text{skip}] ; 2$$

Soundness

The Taylor expansion accurately reflects the operational semantics of SCI:

Theorem. If $M, \sigma \rightarrow M', \sigma'$ then for every $t' \in \text{Val}(\sigma', -)$ and every $m' \in \mathbf{T}(M')$, there is $m \in \mathbf{T}(M)$ and $t \in \text{Val}(\sigma, -)$ such that

$$NF(m\langle t/L \rangle) = NF(m'\langle t'/L' \rangle).$$

This is proved by straightforward induction.

Adequacy

We can prove an adequacy result: if the Taylor expansion of a closed program is non-zero, the program terminates.

Theorem. For a program $\vdash^\emptyset M : \mathbf{U}$, if $NF(\mathbf{T}(M)) \neq \mathbf{0}$ then $M, \sigma \rightarrow^* \text{skip}, \sigma'$, where σ is the unique state on no locations and σ' is some state.

The proof is a typical logical relations argument.

Combining soundness and adequacy with the compositionality of the Taylor expansion we obtain:

Theorem. For terms $\Gamma \vdash^L M, N : A$ of SCI, if $NF(\mathbf{T}(M)) = NF(\mathbf{T}(N))$ then M and N are observably equivalent.

However, the Taylor expansion is *not* a fully abstract semantics.
For example, in SCI

$$\lambda x. \lambda y. x ; y \cong \lambda x. \lambda y. y ; x : \mathbf{U} \rightarrow \mathbf{U} \rightarrow \mathbf{U}$$

but their Taylor expansions are clearly different.

Summary

We introduced a noncommutative *resource calculus* which serves as a target of a Taylor expansion of imperative programs.

The calculus is confluent, strongly normalizing, and has no reduction rules for handling imperative behaviour!

Normal forms of the calculus can be seen as basic bricks of *approximants* to SCI programs.

- not discussed: eta-long normal forms denote single elements of Reddy's relational model

Directions

- Can we characterise Taylor equivalence? What does it mean for $NF(\mathbf{T}(M)) = NF(\mathbf{T}(N))$?
- Passive types: Reynolds's SCI allowed sharing of identifiers bound to phrases which do not write to the store. Can we develop a hybrid list/multiset resource calculus to incorporate these?
- Beyond SCI: can we handle full Idealized Algol similarly? Game semantics provides models of Algol, and the models are differential categories.
- Would like to understand the relationship between this account of state and others e.g. algebraic effects

Bibliography

- Abramsky, S. and McCusker, G., 1996. Linearity, sharing and state: a fully abstract game semantics for Idealized Algol with active expressions (extended abstract). *Proceedings of 1996 workshop on linear logic*, 3. Electronic notes in theoretical computer science. Elsevier, pp.2–14.
- Barbarossa, D. and Manzonetto, G., 2019. Taylor subsumes Scott, Berry, Kahn and Plotkin. *Proc. acm program. lang.* [Online], 4(POPL). Available from: <https://doi.org/10.1145/3371069>.
- Blute, R.F., Cockett, J.R.B. and Seely, R.A.G., 2006. Differential categories. *Mathematical. structures in comp. sci.* [Online], 16(6), pp.1049–1083. Available from: <https://doi.org/10.1017/S0960129506005676>.
- Ehrhard, T. and Regnier, L., 2003. The differential lambda-calculus. *Theoretical computer science* [Online], 309(1), pp.1–41. Available from: [https://doi.org/https://doi.org/10.1016/S0304-3975\(03\)00392-X](https://doi.org/https://doi.org/10.1016/S0304-3975(03)00392-X).
- Ehrhard, T. and Regnier, L., 2008. Uniformity and the Taylor expansion of ordinary lambda-terms. *Theoretical computer science* [Online], 403(2-3), pp.347–372. Available from: <https://doi.org/10.1016/j.tcs.2008.06.001> [Accessed 29 November 2023].
- Girard, J.-Y., 1988. Normal functors, power series and lambda-calculus. *Ann. pure appl. logic*, 37, pp.129–177.
- Kerjean, M.M. and Pédrot, P.-M., 2024. Δ is for Dialectica. *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science* [Online]. LICS '24. New York, NY, USA: Association for Computing Machinery, pp.1–13. Available from: <https://doi.org/10.1145/3661814.3662106> [Accessed 12 March 2025].
- McCusker, G., 2002. A fully abstract relational model of syntactic control of interference. *Proceedings, computer science logic (CSL) 2002*, 2471. Lecture notes in computer science. Springer-Verlag, pp.247–261.
- O'Hearn, P.W., 1993. A model for syntactic control of interference. *Mathematical structures in computer science*, 3(4), pp.435–465.
- O'Hearn, P.W. and Pym, D.J., 1999. The logic of bunched implications. *Bulletin of symbolic logic*, 5(2), pp.215–244.
- Reddy, U.S., 1996. Global state considered unnecessary: Object-based semantics for interference-free imperative programs. *Lisp and symbolic computation*, 9(1), pp.7–76.
- Reynolds, J.C., 1978. Syntactic control of interference. *Conf. record 5th acm symposium on principles of programming languages*. pp.39–46.
- Reynolds, J.C., 2002. Separation logic: a logic for shared mutable data structures. *Seventeenth annual IEEE symposium on logic in computer science (lics 2002)*. IEEE Computer Society Press, pp.55–74.
- Tsukada, T., Asada, K. and Ong, C.-H.L., 2017. Generalised species of rigid resource terms. *2017 32Nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)* [Online]. pp.1–12. Available from: <https://doi.org/10.1109/LICS.2017.8005093> [Accessed 28 November 2023].

Coda: Reddy's model

The ideas of differential lambda calculus and Taylor expansion grew from *quantitative semantics* of lambda-calculus and linear logic [Girard 1988].

The simplest variant of quantitative semantics is the *relational model*:

- types = sets
- $\llbracket A \rightarrow B \rrbracket = \mathbf{M}_f \llbracket A \rrbracket \times \llbracket B \rrbracket$
- terms $x : A \vdash M : B$ = relations between $\mathbf{M}_f A$ and B

E.g.

$$\llbracket x : \mathbf{Z} \vdash x - x : \mathbf{Z} \rrbracket = \{([n, m], n - m) \mid n, m \in \mathbf{Z}\}.$$

In function application, the argument can provide multiple inputs to the function.

For closed terms:

$$\llbracket MN \rrbracket = \{b \mid ([a_1, \dots, a_k], b) \in \llbracket M \rrbracket, a_1, \dots, a_k \in \llbracket N \rrbracket\}.$$

Note that all inputs are drawn from $\llbracket N \rrbracket$ and the order does not matter.

cf. $\mathbf{T}(M)[\mathbf{T}(N)^k]$.

Resource terms provide finer control over the contents of the input bags.

Interpretation of resource calculus terms:

- closed term of type A = subset of A
- bag of closed terms of type A = subset of $\mathbf{M}_f A$.

$$\llbracket [M_1, \dots, M_n] \rrbracket = \{ [m_1, \dots, m_n] \mid m_i \in \llbracket M_i \rrbracket \}$$

Resource terms describe *building blocks* of the relational semantics.

In some situations, a normal-form resource term denotes a *single element* of the relational model.

Reddy's model of imperative programming

We outline the basic definitions of Reddy's model of SCI, for comparison.

- Types = sets
- $\llbracket A \rightarrow B \rrbracket = \llbracket A \rrbracket^* \times \llbracket B \rrbracket$
- Terms $x : A \vdash M : B$ = relations between A^* and B

The *unit type* $\mathbf{U}$ is modelled as a singleton set $\{*\}$.

Locations are treated as variables of a special type $\mathbf{V}$, modelled as the set of possible *read* and *write* actions.

$$\{r(i) \mid i \in \mathbf{N}\} \cup \{w(i) \mid i \in \mathbf{N}\}$$

The term

$$f : \mathbf{U} \rightarrow \mathbf{U} \vdash^l f(l := !l + 1) ; !l : \mathbf{N}$$

is interpreted as a relation with entries such as

$$\begin{array}{ccccc} \mathbf{U} & \rightarrow & \mathbf{U}, & \mathbf{V} & \vdash & \mathbf{N} \\ f : [([*, *], & & *), & l : [r(i), w(i + 1), r(j), w(j + 1), r(k)], & & k \end{array}$$

Semantics of application

As in the relational model of lambda-calculus, the semantics of application of closed terms is easy to describe:

$$\llbracket MN \rrbracket = \{b \mid ([a_1, \dots, a_k], b) \in \llbracket M \rrbracket, a_1, \dots, a_k \in \llbracket N \rrbracket\}.$$

Semantics of local variables

Semantics of local variable application is similar!

Given

$$\Gamma \vdash^l M : \mathbf{X}$$

we have

$$\llbracket \Gamma \vdash^\emptyset \text{new } l \text{ in } M \rrbracket = \{(s, b) \mid (s, t, b) \in \llbracket M \rrbracket, t \text{ is a valid state trace}\}$$

Recalling our example:

$$\begin{array}{c} \mathbf{U} \quad \rightarrow \quad \mathbf{U}, \quad \mathbf{V} \quad \vdash \quad \mathbf{N} \\ f : [([**], \quad *)], \quad l : [r(i), w(i + 1), r(j), w(j + 1), r(k)], \quad k \end{array}$$

After allocation of l as a local variable, the only elements in play are those with valid state traces

$$\begin{array}{c} \mathbf{U} \quad \rightarrow \quad \mathbf{U}, \quad \mathbf{V} \quad \vdash \quad \mathbf{N} \\ f : [([**], \quad *)], \quad l : [r(0), w(1), r(1), w(2), r(2)], \quad 2 \end{array}$$

and the final semantics is

$$\begin{array}{c} \mathbf{U} \quad \rightarrow \quad \mathbf{U} \quad \vdash \quad \mathbf{N} \\ f : [([**], \quad *)] \quad 2 \end{array}$$